UTEC
Universidad Tecnológica

Maven

¿Qué es Maven?

Maven es una herramienta de gestión de proyectos y un sistema de automatización de compilación utilizado principalmente en proyectos Java. Proporciona una forma estandarizada de gestionar dependencias, compilaciones, informes, documentación y otros aspectos del ciclo de vida de un proyecto de software. El objetivo principal de Maven es simplificar y estandarizar el proceso de construcción y gestión de proyectos, lo que lo convierte en una herramienta clave en el desarrollo de software moderno.

¿Para qué se usa Maven?

1. **Gestión de dependencias**: Maven permite a los desarrolladores declarar las bibliotecas y marcos de terceros que necesitan en su proyecto. Maven se encarga de descargar estas dependencias automáticamente desde repositorios remotos, lo que simplifica el proceso de configuración y uso de bibliotecas externas.
2. **Automatización del ciclo de vida**: Maven ofrece un ciclo de vida de compilación estandarizado. Define fases como compilación, pruebas, empaquetado y despliegue. Los desarrolladores pueden ejecutar un solo comando, como `mvn install`, y Maven se encargará de todo el proceso.
3. **Estandarización**: Proporciona una estructura estandarizada de proyecto. En un proyecto Maven, la disposición de directorios, nombres de archivos y otros elementos están organizados de manera coherente, lo que facilita a los equipos seguir las mismas convenciones.
4. **Multiproyectos**: Permite gestionar proyectos que dependen de otros proyectos, facilitando el trabajo en proyectos grandes con múltiples módulos.

Estructura de un proyecto Maven

Maven organiza los proyectos en un conjunto de directorios y archivos predefinidos. La configuración principal del proyecto se realiza en un archivo llamado `pom.xml` (Project Object Model). Este archivo define el proyecto, sus dependencias y los plugins necesarios para construirlo.

UTEC
Universidad Tecnológica

Ejemplo de un archivo `pom.xml` básico:

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>mi-proyecto</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencies>
        <!-- Ejemplo de dependencia -->
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

Ventajas de usar Maven

1. **Gestión automática de dependencias**: Maven descarga y gestiona automáticamente las bibliotecas necesarias para tu proyecto desde repositorios centrales. Esto ahorra tiempo y esfuerzo, ya que no necesitas gestionar manualmente las dependencias.
2. **Ciclo de vida estandarizado**: El ciclo de vida de compilación predefinido de Maven (compilar, probar, empaquetar, instalar, desplegar) estandariza el proceso de construcción de proyectos, lo que hace que sea más fácil para los equipos de desarrollo seguir las mismas prácticas.
3. **Facilidad de integración**: Maven se integra fácilmente con herramientas de integración continua como Jenkins, lo que permite la automatización de pruebas, despliegues y otros procesos del ciclo de vida del desarrollo.
4. **Portabilidad**: Debido a la estructura y configuración estándar de Maven, es fácil mover un proyecto entre diferentes entornos de desarrollo o entre desarrolladores sin tener que preocuparse por configuraciones personalizadas.

5. **Extensibilidad**: Maven es altamente extensible mediante el uso de plugins. Los plugins permiten a los desarrolladores añadir funcionalidades adicionales, como generar documentación, ejecutar análisis de código, empaquetar aplicaciones, etc.
6. **Reutilización de código y proyectos multiproyecto**: En proyectos grandes, Maven facilita la división en módulos y la reutilización de código entre ellos. Los desarrolladores pueden crear múltiples subproyectos dentro de un mismo proyecto Maven, mejorando la gestión y organización del código.

Ciclo de vida de Maven

Maven sigue un ciclo de vida predefinido que incluye varias fases clave:

- **compile**: Compila el código fuente del proyecto.
- **test**: Ejecuta las pruebas unitarias utilizando un marco como JUnit.
- **package**: Empaqueta el proyecto en un formato distribuible, como un archivo JAR o WAR.
- **install**: Instala el paquete en el repositorio local para ser usado como dependencia en otros proyectos.
- **deploy**: Despliega el proyecto en un repositorio remoto (por ejemplo, un servidor de artefactos como Nexus).

Comandos básicos de Maven

- `mvn compile`: Compila el código fuente del proyecto.
- `mvn test`: Ejecuta las pruebas unitarias.
- `mvn package`: Empaqueta el proyecto en un archivo JAR o WAR.
- `mvn install`: Instala el proyecto en el repositorio local.
- `mvn clean`: Elimina los archivos generados por compilaciones anteriores.
- `mvn dependency:tree`: Muestra la jerarquía de dependencias del proyecto.

Maven es una herramienta poderosa que simplifica la gestión de proyectos Java y permite a los desarrolladores centrarse en escribir código en lugar de preocuparse por la configuración del entorno de compilación o la gestión de dependencias. Su estandarización y ciclo de vida predefinido lo convierten en una herramienta esencial en el desarrollo de software moderno, especialmente para proyectos grandes y complejos.

Ejemplo de uso: conexion a la base de datos

Para crear un proyecto Maven Java básico en **IntelliJ** y agregar la dependencia del driver de PostgreSQL, sigue estos pasos:

UTEC
Universidad Tecnológica

1. Crear un nuevo proyecto Maven

1. Abre **IntelliJ IDEA**.
2. Haz clic en **File** > **New** > **Project**.
3. Selecciona **Maven** en el menú del lado izquierdo.
4. Marca la opción **Create from archetype** para usar un arquetipo predefinido o déjalo sin marcar para empezar desde cero. Luego, haz clic en **Next**.
5. Rellena los campos:
    - **GroupId**: Define el identificador del grupo. Ejemplo: `com.miempresa`.
    - **ArtifactId**: El nombre de tu proyecto. Ejemplo: `proyecto-maven`.
    - **Version**: Deja la versión por defecto o cámbiala si lo prefieres.
6. Haz clic en **Finish**.

2. Agregar la dependencia del driver de PostgreSQL

Una vez que se haya creado el proyecto, ubica y abre el archivo **`pom.xml`** (este archivo gestiona las dependencias y la configuración de Maven).

Dentro de **`pom.xml`**, agrega la siguiente dependencia para el driver de PostgreSQL dentro de la etiqueta `<dependencies>`:

```
<dependencies>
    <!-- Dependencia para el driver de PostgreSQL -->
    <dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
        <version>42.6.0</version> <!-- Puedes revisar la última versión
-->
    </dependency>
</dependencies>
```

Guarda el archivo. IntelliJ debería sincronizar automáticamente el proyecto y descargar la dependencia de PostgreSQL desde el repositorio de Maven.

3. Estructura del proyecto

El proyecto básico Maven que creaste tendrá la siguiente estructura de carpetas:

UTEC
Universidad Tecnológica

```
proyecto-maven/
|
├── src/
|   ├── main/
|   |   └── java/  <-- Aquí colocas tu código Java principal
|   └── test/
|       └── java/  <-- Aquí colocas tus pruebas unitarias
|
└── pom.xml  <-- Archivo de configuración de Maven
```

4. Crear una clase Java de prueba

Dentro de la carpeta **`src/main/java`**, crea una clase Java básica para probar la conexión con PostgreSQL. Por ejemplo:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class TestPostgreSQLConnection {

    public static void main(String[] args) {
        String url =
"jdbc:postgresql://localhost:5432/tu_base_de_datos";
        String user = "tu_usuario";
        String password = "tu_contraseña";

        try {
            Connection connection = DriverManager.getConnection(url,
user, password);
            System.out.println("Conexión exitosa a PostgreSQL!");
        } catch (SQLException e) {
            System.out.println("Error al conectarse a PostgreSQL: " +
e.getMessage());
        }
    }
}
```

Universidad Tecnológica

Reemplaza `localhost`, `tu_base_de_datos`, `tu_usuario`, y `tu_contraseña` con los valores correspondientes a tu configuración de base de datos.

5. Ejecutar el proyecto

Haz clic derecho sobre la clase `TestPostgreSQLConnection` y selecciona **Run 'TestPostgreSQLConnection.main()'**.

Deberías ver en la consola un mensaje indicando si la conexión a la base de datos fue exitosa o si hubo algún error.